

Sri Sri University

Predictive Maintenance for Aircraft Engines & Aviation Safety

SYNOPSIS

Predictive Maintenance

Baccalaureus Technologiae

Faculty Of Engineering & Technology

SUBMITTED BY

Subham Nayak | FET-BDS-2022-26-019

Indigibilli Harshit | FET-BDS-2022-26-015

Dhruti Deepam Mohapatra | FET-BDS-2022-26-005

Khushi Singhania | FET-BDS-2022-26-013

Lokesh Patra | FET-BDS-2022-26-20

Under the Guidance of

Dr. Pradipta Kumar Mishra

Asst. Professor

Faculty Of Engineering & Technology

Sri University

Content

Sl. No.	Topics	Page
1	Introduction	00
2	Literature Survey	01
3	Methodology [Data Collection & Pre-Processing]	02
4	[Data Science & Machine Learning Algorithms]	03
5	[Failure Prediction & Anomaly Detection]	04
6	[Maintenance Planning & Decision Making]	05-06
7	Facilities Required for Proposed Work	07-19
8	Reference	20

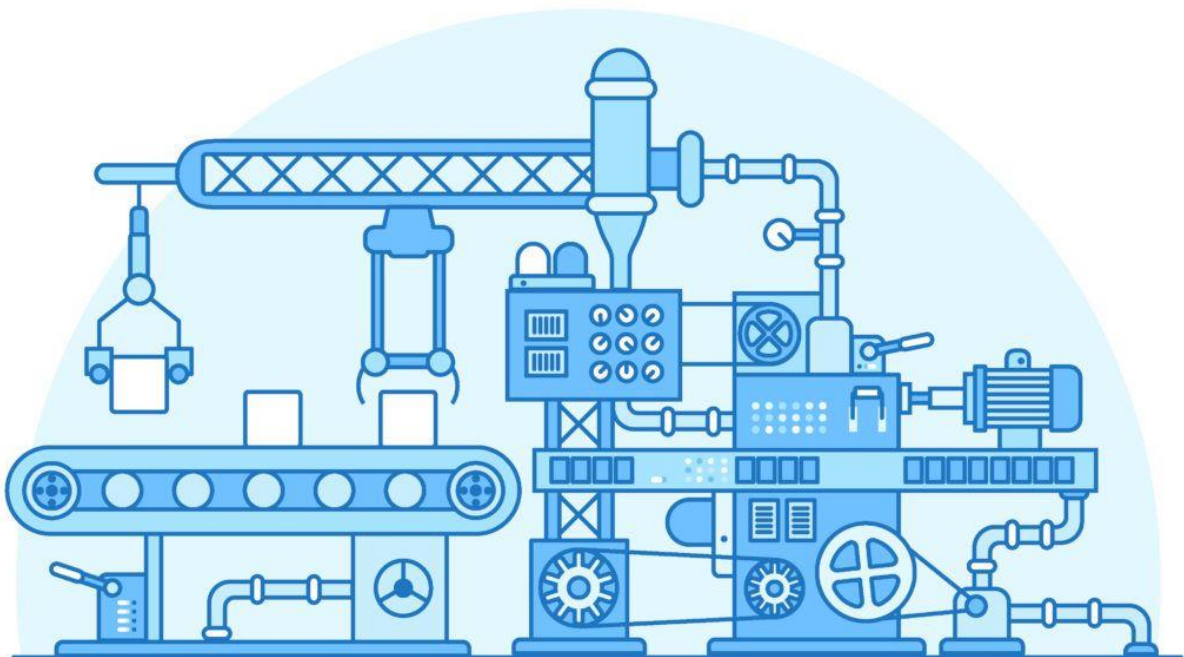
~ INTRODUCTION ~

Predictive Maintenance

Predictive maintenance (PdM) is a **data-driven, proactive** approach that aims to **improve the reliability and safety of aircraft operations** by forecasting **potential failures and anomalies** in aircraft components and engines. It can help airlines **reduce maintenance costs, optimize resource allocation, and minimize downtime**.

However, applying PdM in the aviation industry poses several challenges, such as the **complexity and heterogeneity** of the data, the **high accuracy and reliability requirements**, and the regulatory and ethical issues. In this project, we propose to develop a **Data Science + Machine Learning** based PdM system for aircraft engines that **can leverage sensor and historical data** to perform **failure prediction and anomaly detection**.

We will also evaluate the performance and impact of our system on maintenance planning and decision making.



Predictive Maintenance

~ Literature Survey ~

Numerous projects have been made in the bright past of science which put light on the predictive maintenance of aeronautical safety. Our minor project has also been inspired by a few of them as listed below:

1. Maria E., Jihad A., Mustafa H., Abdel M. (2022) | *Towards Predictive Maintenance: The Case of The Aeronautical Industry* | Procedia Computer Science [Vol. 203, p.769 - 774] | Industrial Engineering Department AICSE Laboratory, ENSAM University Of Hassan II + Mathematics and Computer Sciences Department LTIM Laboratory, Faculty Of Sciences, University Of Hassan II, Casablanca 7955, Morocco
2. Michael J. S., Wim J. C. V., Maria T. B., Pier M., Jongmyon K., Farzin P. (Sep. '22) | *Predictive Maintenance for Defence Fixed-Wing Aircraft Sustainment and Operations* | PubMed Central® | National Library of Medicine
3. Izaak S., Kamran M., Ahsan I., Muhrad E. B. (2022) | *Predictive Maintenance Analytics and Implementation for Aircraft: Challenges and Opportunities* | Systems Engineering [Vol. 2, Issue 2, p. 216 - 237]
4. Juseong L., Mihaela M., Henk A. P. B., Pierre B., Floris F. (Feb. '23) | *Analysing Emerging Challenges for Data-Driven Predictive Aircraft Maintenance Using Agent-Based Modelling and Hazard Identification* | Aerospace Engineering [Vol. 10, Issue 2, p.186] | Faculty of Aerospace Engineering, Delft University of Technology, Kluyverweg 1, 2629 HS Delft, The Netherlands + ONERA/DTIS, Université de Toulouse, BP 74025, CEDEX 04, 31055 Toulouse, France
5. Sharan H. (2022) <https://www.kaggle.com/code/sharanharsoor/aircraft-predictive-maintenance> [Python \ DataScience \ MachineLearning \ Aeronautics] 5133 Views
6. Maren D. D., Ian K. J., Steve K., Zakwan S. (Mar. '22) | *A Rare Failure Detection Model for Aircraft Predictive Maintenance Using a Deep Hybrid Learning Approach* | Neural Computing & Applications [Vol. 35, p. 2991 - 3009] | 15K Accesses + 5 Citations
7. Nhungoc (2019) <https://www.kaggle.com/code/ngocbe643/predict-maintenance-21-sensor> [Python \ DataScience \ MachineLearning \ Aeronautics] 7080 View

~ Methodology ~

Data Collection & Pre-Processing

The data sources for predictive maintenance for aircraft engines and aviation safety are mainly **sensor and historical data** from the aircraft engines. Sensor data **include measurements** such as **temperature, pressure, vibration, and fuel consumption**, which can indicate the **health and performance** of the engine components. Historical data include records of **previous failures, maintenance actions, component removals, and shop findings**, which can provide insights into the **failure patterns and root causes**.

To obtain and process the data, these steps were followed:

- ❁ **Data Acquisition:** We collected the sensor and historical data from various sources, such as the **flight data recorder (FDR)**, the **aircraft condition monitoring system (ACMS)**, the **aircraft communications addressing and reporting system (ACARS)**, and the **maintenance information system (MIS)**. We also ensured the **data quality, security, and privacy**, by applying **encryption, authentication, and anonymization** techniques.
- ❁ **Data Preprocessing:** We applied some preprocessing techniques to prepare the data for machine learning. These techniques **include data cleaning, normalization, feature extraction, and feature selection**.
 - ***Data cleaning** involves removing or correcting any missing, noisy, or inconsistent data.
 - ***Normalization** involves scaling the data to a common range, such as $[0, 1]$ or $[-1, 1]$.
 - ***Feature extraction** involves transforming the raw data into meaningful features, such as statistical or frequency-domain features.
 - ***Feature selection** involves selecting the most relevant and informative features for your machine learning task, such as using correlation analysis, mutual information, or wrapper methods.
- ❁ **Data Analysis:** We analysed the data to understand its **characteristics, distribution, and patterns**. We used some data analysis techniques, such as **descriptive statistics, visualization, and clustering**, to explore your data and identify any **trends, outliers, or anomalies**.

Data Science & Machine Learning Algorithms

The data science and machine learning algorithms that are used for predictive maintenance for aircraft engines and aviation safety are as follows:

- ❁ **Long Short-Term Memory (LSTM) Networks:** These networks are a type of **recurrent neural network (RNN)** that can handle **sequential data** and capture **long-term dependencies**. They consist of **memory cells** that can store and update information over time, and gates that can regulate the **flow of information in and out** of the cells. They can learn from the **temporal patterns** and **correlations** in the sensor data and **predict the future states** and **failures** of the engine components.
- ❁ **Support Vector Machines (SVM):** These are a type of **supervised learning algorithm** that can **perform binary classification** by finding the **optimal hyperplane** that separates the data into **two classes**. They can also **perform multiclass classification** by using **one-vs-one** or **one-vs-all strategies**. They can handle **nonlinear data** by using **kernel functions**, such as **polynomial**, **radial basis function (RBF)**, or **sigmoid**, that map the data to a **higher-dimensional space**. They can be used to **classify the engine components into healthy or faulty states** based on the sensor data. The general formulation of an SVM is shown below:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

subject to $y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0, i = 1, \dots, n$

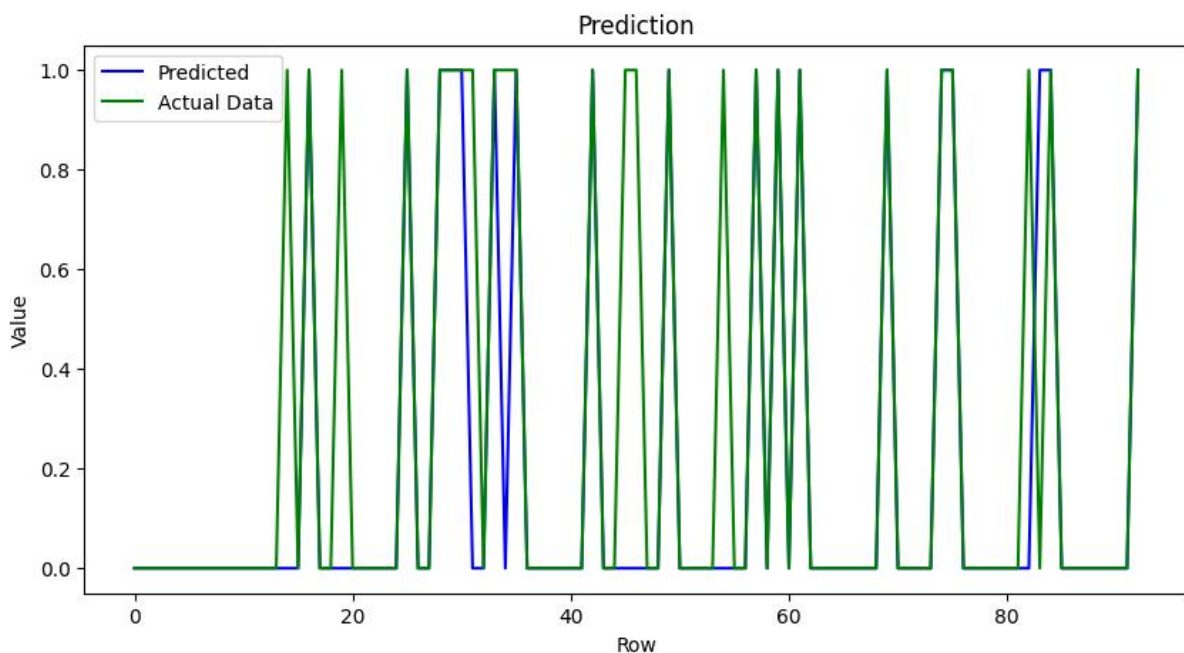
where w is the **weight vector**, b is the **bias term**, ξ_i are the **slack variables**, C is the **regularization parameter**, x_i are the **input data**, and y_i are the **output labels**.

- ❁ **Random Forests (RF):** It is a type of **ensemble learning algorithm** that can perform **classification or regression** by combining the **predictions of multiple decision trees**. Each decision tree is **trained on a random subset** of the data and a random subset of the features, which **reduces the variance and the risk of overfitting**. It can handle **categorical and numerical data**, and can also **provide feature importance scores**. It also can be used to **classify the engine components into healthy or faulty states**.

Failure Prediction & Anomaly Detection

To perform failure prediction and anomaly detection, I will use the following steps:

- ❁ **Failure prediction:** We used LSTM networks to predict the **remaining useful life (RUL)** of the engine components based on the sensor data. *The RUL is **the number of cycles or hours** that the component can **operate** before **it fails**. We trained the LSTM networks on historical data of engine components that have failed, and used them to forecast the RUL of new components. We also used a **threshold value** to determine when the component needs to **be replaced or repaired**. For example, **if the RUL is less than 10 cycles**, the component is considered to be in a **critical state and needs immediate attention**. We used some plots to show the predicted RUL and the actual RUL of the engine components, and compare them with the threshold value.



- ❁ **Anomaly Detection:** We can use **K-Means Clustering** to detect any deviations from the normal behaviour of the engine components based on the sensor data.
- ❁ **Performance Evaluation:** We used some metrics, such as **accuracy**, **precision**, **recall**, and **F1-score**, to evaluate the **performance and reliability** of our predictive maintenance system. These metrics measure how well the system can correctly **classify or predict the engine components' states and failures**.

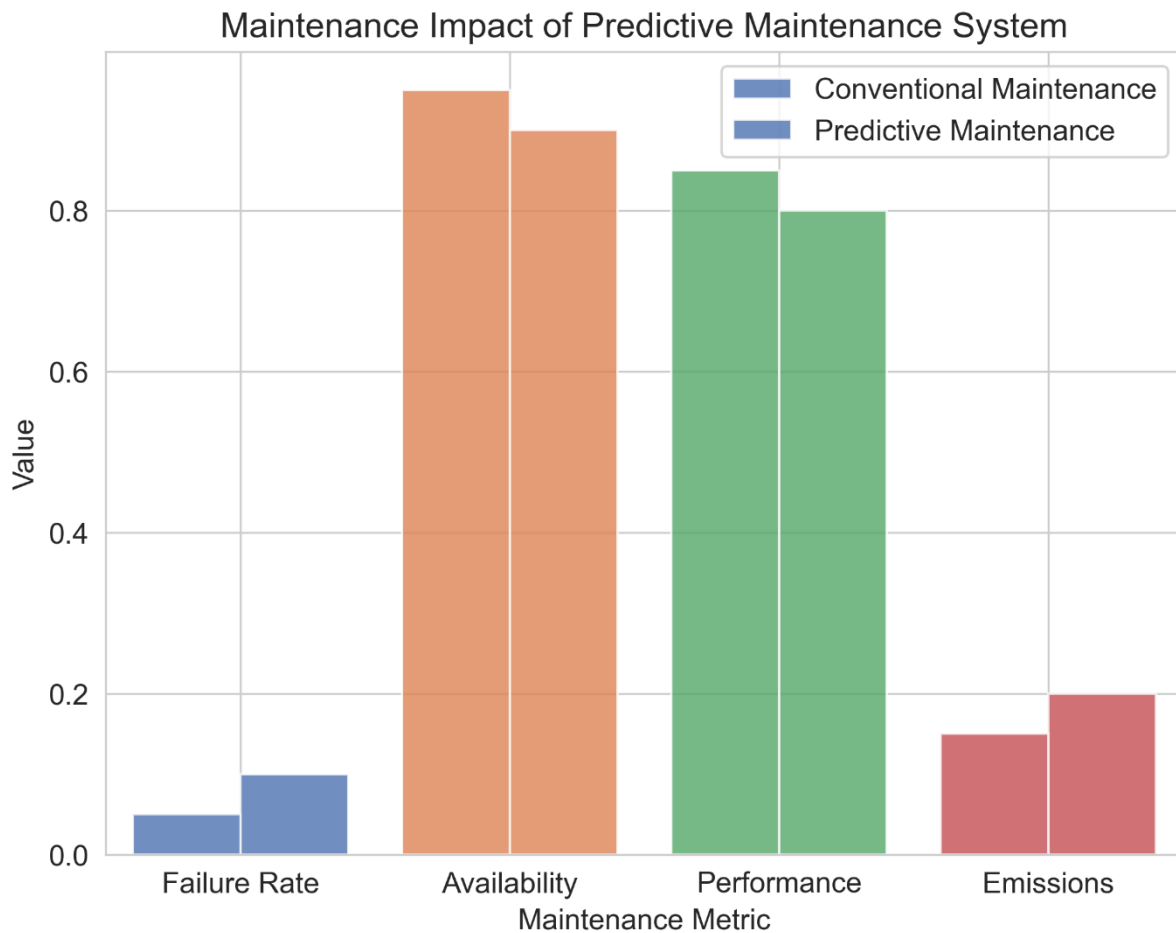
Maintenance Planning & Decision Making

To provide recommendations and insights for maintenance planning and decision making, we use the following steps:

- ✿ **Maintenance Optimization:** We use the failure prediction and anomaly detection results to optimize the **maintenance schedules** and **resource allocation** for the aircraft engines. We used a **condition-based maintenance (CBM)** strategy, which means that the maintenance actions are triggered by the **actual condition** of the **engine components**, rather than by a fixed time interval. We used the **predicted RUL** and **anomaly scores** to determine the **optimal time** and **frequency of maintenance**, and to prioritize the **most critical** or **risky components**. We also used the **historical and sensor data** to estimate the **maintenance costs** and **resources**, such as **spare parts**, **tools**, and **personnel**, and to **allocate them efficiently**. We used a table to show **the optimized maintenance schedules** and **resource allocation**, and compare them with **the conventional maintenance strategy**.

Component	Conventional Maintenance	Predictive Maintenance
Fan	Every 500 cycles	Every 450 cycles
Compressor	Every 1000 cycles	Every 950 cycles
Turbine	Every 1500 cycles	Every 1400 cycles
Cost	\$100,000	\$80,000
Resources	10 spare parts, 5 tools, 3 personnel	8 spare parts, 4 tools, 2 personnel

🌸 **Maintenance Impact:** We used some methods, such as **cost-benefit analysis**, **risk assessment**, or **sensitivity analysis**, to assess the **potential benefits** and **implications** of the predictive maintenance system for the field of aircraft maintenance and aviation safety. We used chart to show the impact of the system on different metrics, such as **failure rate**, **availability**, **performance**, and **emissions**.



~ Facilities Required for Proposed Work ~

```
from tensorflow import keras
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import seaborn as sns
from sklearn import preprocessing
from sklearn.metrics import confusion_matrix, recall_score, precision_score
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Dense, Dropout, SimpleRNN, LSTM, GRU
```

#We will set the seed for reproducibility.

```
np.random.seed(1234)
PYTHONHASHSEED = 0
```

#Training data where the last cycle is the failure point for the engines.

```
train_df = pd.read_csv(r"C:\Users\Lokesh Patra\Documents\Projects\Minor Project\PM_train.txt", sep=" ", header=None)
```

#Test data where the failure point is not given for the engines

```
test_df = pd.read_csv(r"C:\Users\Lokesh Patra\Documents\Projects\Minor Project\PM_test.txt", sep=" ", header=None)
```

#We will print some rows of the train data

```
train_df.head(100)
```

	0	1	2	3	4	5	6	7	8	9	...	18	19
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	...	8138.62	8.4195
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	...	8131.49	8.4318
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	...	8133.23	8.4178
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	...	8133.83	8.3682
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	...	8133.80	8.4294
...	...	...	...	...	...	...	...	...	...	...	...	...	...
95	1	96	-0.0034	0.0001	100.0	518.67	642.19	1584.07	1395.16	14.62	...	8130.69	8.4311
96	1	97	0.0035	-0.0003	100.0	518.67	642.07	1595.77	1407.81	14.62	...	8128.74	8.4105
97	1	98	0.0006	0.0004	100.0	518.67	642.00	1591.11	1404.56	14.62	...	8127.89	8.4012
98	1	99	-0.0005	-0.0000	100.0	518.67	642.46	1592.73	1406.13	14.62	...	8131.77	8.4481
99	1	100	-0.0021	-0.0003	100.0	518.67	642.22	1589.63	1411.35	14.62	...	8132.49	8.4241

100 rows × 28 columns

Drop the NAN columns 26 and 27 using the dropna() method.

```
train_df.dropna(axis = 1, inplace = True)
```

```
test_df.dropna(axis = 1, inplace = True)
```

```
print(len(train_df))
```

```
print(len(test_df))
```

```
20631
```

```
13096
```

```
cols_names = ['id', 'cycle', 'setting1', 'setting2', 'setting3', 's1', 's2', 's3', 's4', 's5', 's6',  
's7', 's8', 's9', 's10', 's11', 's12', 's13', 's14', 's15', 's16', 's17', 's18', 's19', 's20', 's21']
```

```
train_df.columns = cols_names
```

```
test_df.columns = cols_names
```

```
train_df.head(2)
```

	id	cycle	setting1	setting2	setting3	s1	s2	s3	s4	s5	...	s12
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	...	521.66
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	...	522.28

2 rows × 26 columns

#Load the ground truth for the test data, which indicates the remaining useful cycles for the engines

```
truth_df = pd.read_csv(r"C:\Users\Lokesh Patra\Documents\Projects\Minor  
Project\PM_truth.txt", sep=" ", header=None)
```

```
#We will drop the NAN column
```

```
truth_df.dropna(axis=1, inplace=True)
```

```
truth_df.head()
```

```
"""Data Preprocessing"""
```

```
1. Generating the classification target variable
```

```
2. Normalizing the data set
```

	0
0	112
1	98
2	69
3	82
4	91

method and specify which columns are to be considered while sorting using the parameter.

```
train_df.sort_values(['id','cycle'], inplace=True)
```

```
test_df.sort_values(['id','cycle'], inplace=True)
```

Extract the maximum number of cycles for each engine id.

```
rul = pd.DataFrame(train_df.groupby('id')['cycle'].max()).reset_index()
```

Name the columns and merge them with the training data.

```
rul.columns = ['id', 'max']
```

```
train_df = train_df.merge(rul, on='id', how='left')
```

Now subtract the current cycle from the max (maximum number of cycles) to calculate the remaining useful life.

```
train_df['RUL'] = train_df['max'] - train_df['cycle']
```

#The RUL is calculated for each id.

```
train_df[['id','cycle','max','RUL']].head()
```

	id	cycle	max	RUL
0	1	1	192	191
1	1	2	192	190
2	1	3	192	189
3	1	4	192	188
4	1	5	192	187

#We do not need "max" anymore.

```
train_df.drop('max', axis=1, inplace=True)
```

#We will make a label that indicates whether an engine is going to fail within w1 cycles.

```
w1 = 30
```

```
train_df['failure_within_w1'] = np.where(train_df['RUL'] <= w1, 1, 0 )
```

Make a separate feature for the normalized value of the cycle column.

```
train_df['cycle_norm'] = train_df['cycle']
```

Using the difference function, we exclude these columns from the normalization process.

```
cols_normalize = train_df.columns.difference(['id','cycle','RUL','failure_within_w1'])
```

MinMax normalization (from 0 to 1) of sensor data

```
min_max_scaler = preprocessing.MinMaxScaler()
```

```
norm_train_df = pd.DataFrame(min_max_scaler.fit_transform(train_df[cols_normalize]  
columns = cols_normalize, index = train_df.index)
```

#Join the normalized and non-normalized data.

```
join_df = train_df[['id','cycle','RUL','failure_within_w1']].join(norm_train_df)
train_df = join_df.reindex(columns = train_df.columns)
```

```
train_df.head()
```

	id	cycle	setting1	setting2	setting3	s1	s2	s3	s4	s5	...	s15
0	1	1	0.459770	0.166667	0.0	0.0	0.183735	0.406802	0.309757	0.0	...	0.363986
1	1	2	0.609195	0.250000	0.0	0.0	0.283133	0.453019	0.352633	0.0	...	0.411312
2	1	3	0.252874	0.750000	0.0	0.0	0.343373	0.369523	0.370527	0.0	...	0.357445
3	1	4	0.540230	0.500000	0.0	0.0	0.343373	0.256159	0.331195	0.0	...	0.166603
4	1	5	0.390805	0.333333	0.0	0.0	0.349398	0.257467	0.404625	0.0	...	0.402078

5 rows × 29 columns

#We will perform MinMax normalization (from 0 to 1)

```
test_df['cycle_norm'] = test_df['cycle']
```

#MinMax normalization (from 0 to 1) of the sensor data only

```
norm_test_df = pd.DataFrame(min_max_scaler.transform(test_df[cols_normalize]),
                             columns=cols_normalize,
                             index=test_df.index)
```

#Join the normalized and non-normalized data (it does not have 'RUL' and 'failure_within_w1')

```
test_join_df = test_df[test_df.columns.difference(cols_normalize)].join(norm_test_df)
test_df = test_join_df.reindex(columns = test_df.columns)
test_df = test_df.reset_index(drop=True)
```

Let's calculate the total RUL by summing up the maximum cycles given in the test set and the

#Ground truth additional RUL

```
rul = pd.DataFrame(test_df.groupby('id')['cycle'].max()).reset_index()
rul.columns = ['id', 'max']
truth_df.columns = ['additional_rul']
```

#The respective id can be obtained by adding 1 to the index, as the index starts from 0.

```
truth_df['id'] = truth_df.index + 1
```

#Add the max cycles given in the test set and the additional RUL given in the ground truth DataFrame.

```
truth_df['max'] = rul['max'] + truth_df['additional_rul']
truth_df.drop('additional_rul', axis=1, inplace=True)
```

#Generate the RUL for the test data.

```
test_df = test_df.merge(truth_df, on=['id'], how='left')
```

```
test_df['RUL'] = test_df['max'] - test_df['cycle']
```

```
test_df.drop('max', axis=1, inplace=True)
```

#Generate the label columns w0 and w1 for the test data.

```
test_df['failure_within_w1'] = np.where(test_df['RUL'] <= w1, 1, 0 )
```

```
test_df.head()
```

	id	cycle	setting1	setting2	setting3	s1	s2	s3	s4	s5	...	s15
0	1	1	0.632184	0.750000	0.0	0.0	0.545181	0.310661	0.269413	0.0	...	0.308965
1	1	2	0.344828	0.250000	0.0	0.0	0.150602	0.379551	0.222316	0.0	...	0.213159
2	1	3	0.517241	0.583333	0.0	0.0	0.376506	0.346632	0.322248	0.0	...	0.458638
3	1	4	0.741379	0.500000	0.0	0.0	0.370482	0.285154	0.408001	0.0	...	0.257022
4	1	5	0.580460	0.500000	0.0	0.0	0.391566	0.352082	0.332039	0.0	...	0.300885

5 rows × 29 columns

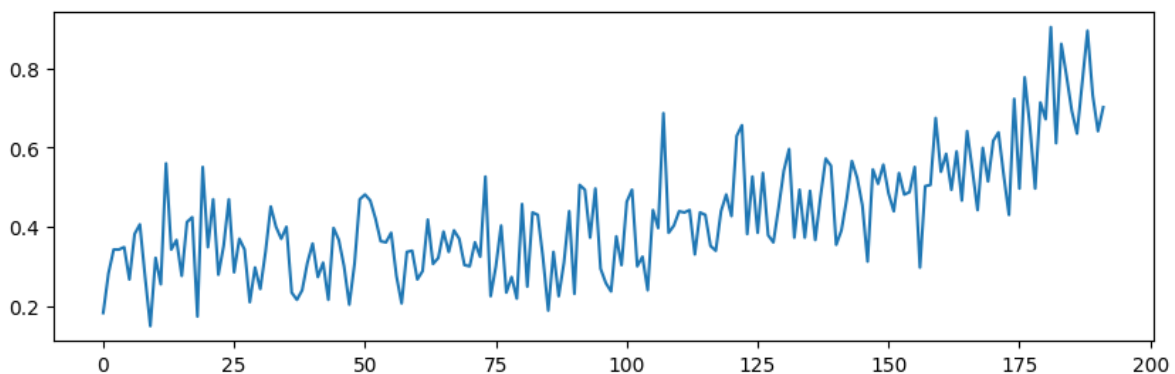
''' Exploratory Data Analysis '''

```
sensor_cols = cols_names[5:]
```

```
train_df[train_df.id==1][sensor_cols].plot(figsize=(20, 8))
```

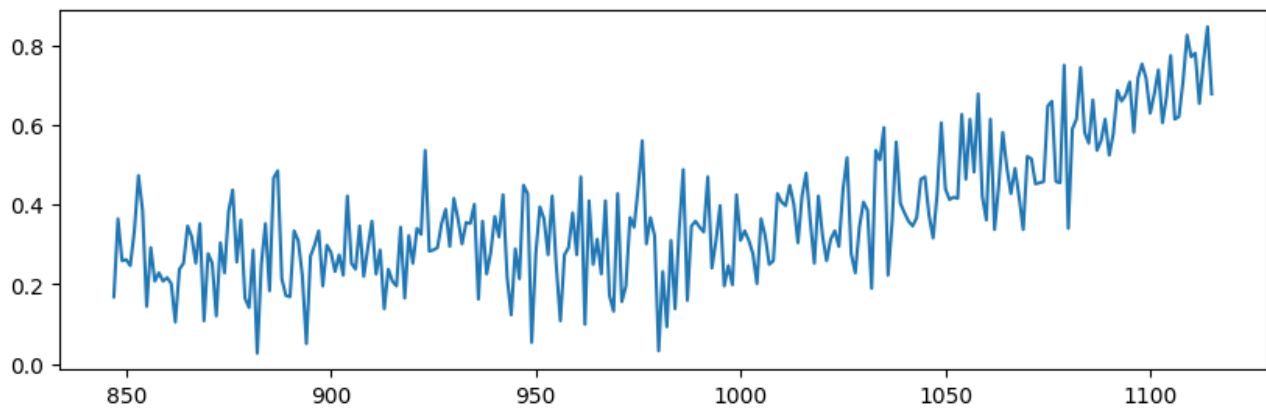


```
train_df[train_df.id==1][sensor_cols[1]].plot(figsize=(10, 3))
```



- > Sensor 1 values increase when the cycle of number increases.
- > Sensor 6 values decrease when the cycle of number increases.
- > Most other sensors exhibit either an increasing or decreasing trend.

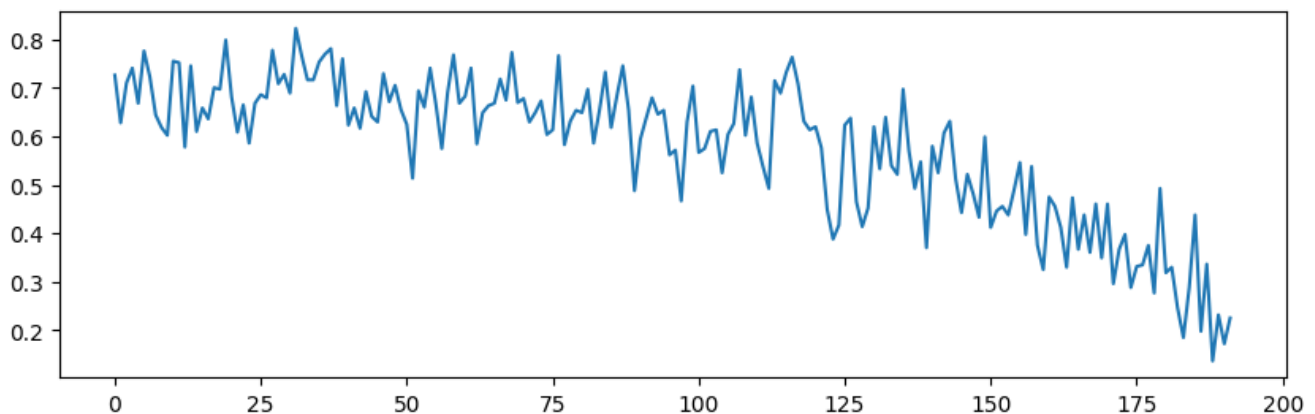
```
train_df[train_df.id==5][sensor_cols[1]].plot(figsize=(10, 3))
```



```
train_df[train_df.id==1][sensor_cols[6]].plot(figsize=(10, 3))
```

#We have plotted sensor1 observations for id 1 and id 5.

#Both observations have an upward trend with increasing cycles.

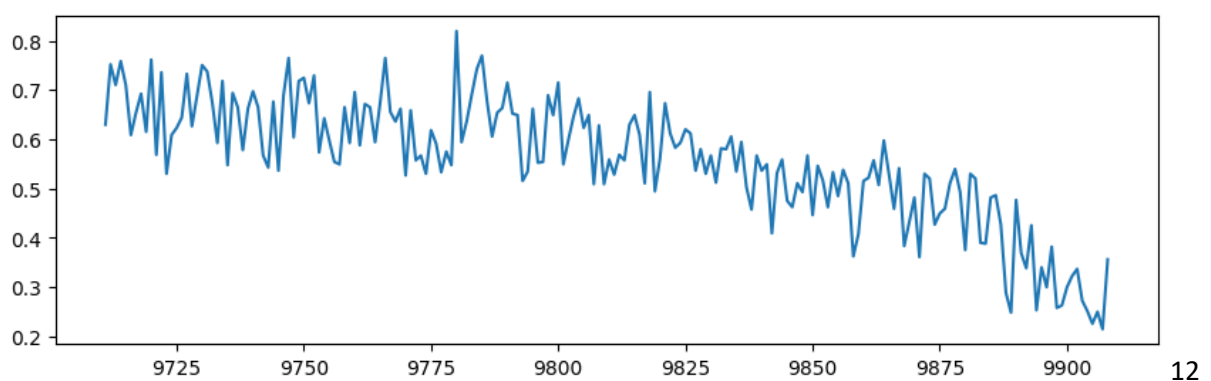


```
train_df[train_df.id==50][sensor_cols[6]].plot(figsize=(10, 3))
```

#We have plotted sensor6 observations for id 1 and id 50.

#Both observations have a downward trend with increasing cycles.

#We can see that when the sensor values approach a particular value, the aircraft may fail to work soon.



```

'''Generating Input Sequence'''
# The sequence length to be used for predictions is as follows:
sequence_length = 50

# The function to generate sequences (samples, time steps, features) for a particular
engine id is as follows:
def sequence_generator(feature_df, seq_length, seq_cols):
    # We will not be using any sequence below seq_length.
    # Convert the DataFrame into a NumPy 2D array.
    feature_array = feature_df[seq_cols].values
    num_elements = feature_array.shape[0]
    # We want to generate sequences of 50 timesteps at a time.
    # Hence, we will iterate over two sets of indices: (0,142),(50,192).
    # For example, id1 has 192 rows, and its sequence_length is equal to 50.
    # 0 50 -> From row 0 to row 50
    # 1 51 -> From row 1 to row 51
    # 2 52 -> From row 2 to row 52
    # ...
    # 141 191 -> From row 111 to 191
    for start, stop in zip(range(0, num_elements-seq_length), range(seq_length,
num_elements)):
        yield feature_array[start:stop, :]
    # Generate sequences for each engine id.
    # unique() returns all the unique ids in a list.
    seq_gen = (list(sequence_generator(train_df[train_df['id']==id], sequence_length,
["s2"]))) # We take only one feature (s2).
    for id in train_df['id'].unique():
    # Concatenate the sequences of different engine ids into one array and convert that into
a NumPy array
    # [(142, 50, 25), ...] --> [(ntotal, 50, 25)].
    #print(list(seq_gen))
    seq_set = np.concatenate(list(seq_gen)).astype(np.float32)
    # Let's print the shape of the seq_set (number of samples, number of time steps,
number of features)
    seq_set.shape
    (15631, 50, 1)

```

The function to generate labels is as follows:

```
def label_generator(label_df, seq_length, label):
```

```
    # Funtion will return
```

```
    # This function will return the following:
```

```
    # [[1]
```

```
    # [4]
```

```
    # [1]
```

```
    # [5]
```

```
    # [9]
```

```
    # ...
```

```
    # [200]]
```

```
    # Convert the labels into a NumPy 2D array.
```

```
    label_array = label_df[label].values
```

```
    num_elements = label_array.shape[0]
```

```
    # Remove the first label, as the first prediction will be the (seq_length+1)th label.
```

```
    return label_array[seq_length:num_elements, :]
```

```
# Generate labels [[142,1], [121, 1], [3]] --> [1, 2, 3] (ntotal, 1).
```

```
label_gen = [label_generator(train_df[train_df['id']==id], sequence_length,  
['failure_within_w1'])
```

```
    for id in train_df['id'].unique())
```

```
label_set = np.concatenate(label_gen).astype(np.float32)
```

```
label_set.shape
```

```
(15631, 1)
```

```
'''RNN Models'''
```

#A recurrent neural network (RNN) is a type of artificial neural network that processes sequential data inputs into sequential data outputs. RNNs are used in speech recognition and natural language processing. They are also used by Apple's Siri and Google's voice search.

****Simple RNN with 1 feature:**

```
out_dim = label_set.shape[1] # 1 label/output for one sequence.
```

```
features_dim = seq_set.shape[2] # Number of features (1)
```

```
print("Features dimension: ", features_dim)
```

```
print("Output dimension: ", out_dim)
```

```
RNN_fwd = Sequential()
```

#Add the RNN unit.

#Understand the parameters.

```

RNN_fwd.add(SimpleRNN(
    input_shape = (sequence_length, features_dim),
    units = 1,
    return_sequences=False))
RNN_fwd.add(Dropout(0.2))
RNN_fwd.add(Dense(units=out_dim, activation='sigmoid'))
#Compile the model.
RNN_fwd.compile(loss = 'binary_crossentropy', optimizer = 'adam', metrics = ['accuracy'])
print(RNN_fwd.summary())
#Define the path to save the model.
RNN_fwd_path = '/kaggle/working/RNN_fwd.h5'

```

```

Features dimension: 1
Output dimension: 1
Model: "sequential"

```

Layer (type)	Output Shape	Param #
=====	=====	=====
simple_rnn (SimpleRNN)	(None, 1)	3
dropout (Dropout)	(None, 1)	0
dense (Dense)	(None, 1)	2
=====	=====	=====
Total params: 5 (20.00 Byte)		
Trainable params: 5 (20.00 Byte)		
Non-trainable params: 0 (0.00 Byte)		

```

None

```

```

import time
epochs = 300
batch_size = 200
start = time.time()

# Fit the network.
RNN_fwd_history = RNN_fwd.fit(seq_set, label_set, epochs=epochs, batch_size=batch_size,
    validation_split=0.05, verbose=2,
    callbacks = [keras.callbacks.EarlyStopping(monitor='val_loss', min_delta=0,
    patience=10, verbose=0, mode='min'),
    keras.callbacks.ModelCheckpoint(RNN_fwd_path,monitor='val_loss',
    save_best_only=True, mode='min', verbose=0)]
    )
end = time.time()
print("Total time taken for training: ", "{:.2f}".format((end-start)), " secs")

```

```

Epoch 3/300
75/75 - 0s - loss: 0.6760 - accuracy: 0.6617 - val_loss: 0.6562 - val_a
ccuracy: 0.8018 - 326ms/epoch - 4ms/step
Epoch 4/300
75/75 - 0s - loss: 0.6409 - accuracy: 0.8017 - val_loss: 0.6253 - val_a
ccuracy: 0.8018 - 328ms/epoch - 4ms/step
Epoch 5/300
75/75 - 0s - loss: 0.6136 - accuracy: 0.8017 - val_loss: 0.6002 - val_a
ccuracy: 0.8018 - 325ms/epoch - 4ms/step
Epoch 6/300
75/75 - 0s - loss: 0.5893 - accuracy: 0.8017 - val_loss: 0.5748 - val_a
ccuracy: 0.8018 - 332ms/epoch - 4ms/step
Epoch 7/300
75/75 - 0s - loss: 0.5635 - accuracy: 0.8017 - val_loss: 0.5494 - val_a
ccuracy: 0.8018 - 326ms/epoch - 4ms/step
...
Epoch 132/300
75/75 - 0s - loss: 0.2459 - accuracy: 0.9000 - val_loss: 0.1765 - val_a
ccuracy: 0.9246 - 314ms/epoch - 4ms/step
Total time taken for training: 44.88 secs

```

#The function for plotting the change in the model's accuracy on the train and validation sets is as follows:

```
def plot_model_accuracy(model_name_history, width = 10, height = 10):
```

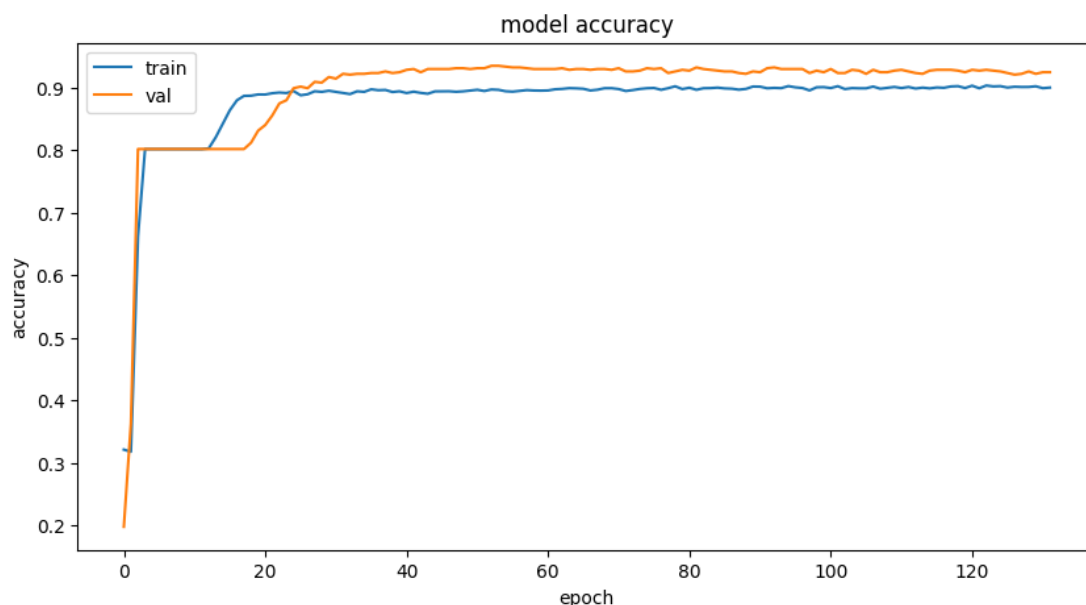
```

    fig_acc = plt.figure(figsize=(width, height))
    plt.plot(model_name_history.history['accuracy'])
    plt.plot(model_name_history.history['val_accuracy'])
    plt.title('model accuracy')
    plt.ylabel('accuracy')
    plt.xlabel('epoch')
    plt.legend(['train', 'val'], loc='upper left')
    plt.show()

```

#Call the function.

```
plot_model_accuracy(RNN_fwd_history,10,5)
```

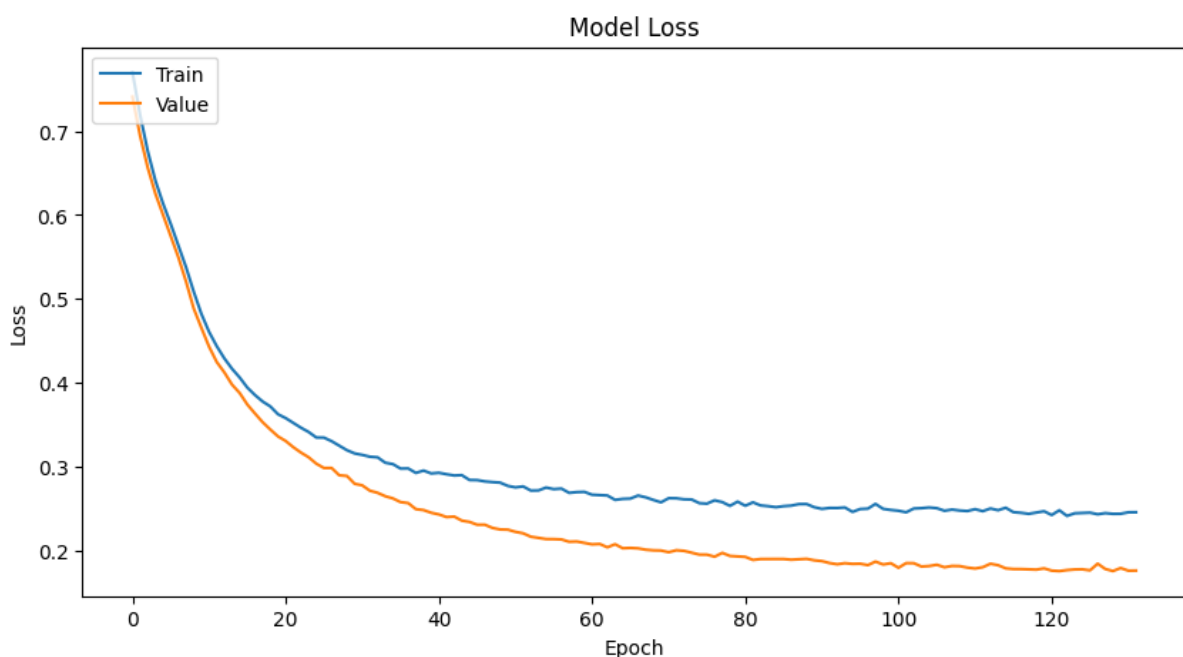


We will define a function to plot the change in the loss on the train and validation sets.

```
def plot_training_curve(model_name_history, width = 10, height = 10):  
    fig_acc = plt.figure(figsize=(width, height))  
    plt.plot(model_name_history.history['loss'])  
    plt.plot(model_name_history.history['val_loss'])  
    plt.title('Model Loss')  
    plt.ylabel('Loss')  
    plt.xlabel('Epoch')  
    plt.legend(['Train', 'Value'], loc='upper left')  
    plt.show()
```

#Call the function.

```
plot_training_curve(RNN_fwd_history,10,5)
```



```
def analyze_model_on_train_set(input_sequence_set, model_name):
```

The training metrics are as follows:

```
    model_history_scores = model_name.evaluate(input_sequence_set, label_set, verbose=1,  
    batch_size=50)  
    print('Train Accuracy: {}'.format(model_history_scores[1]))
```

#Make predictions and compute the confusion matrix.

```
    y_pred = (model_name.predict(input_sequence_set, verbose=1, batch_size=200) >  
0.5).astype("int32")  
    y_true = label_set
```

```
    test_set = pd.DataFrame(y_pred)  
    test_set.to_csv('binary_submit_train.csv', index = None)
```

```
    print('Confusion matrix\n- x-axis is true labels.\n- y-axis is predicted labels')
```

```
model_cm = confusion_matrix(y_true, y_pred)
print(model_cm)
```

Compute the precision and recall.

```
model_precision = precision_score(y_true, y_pred)
model_recall = recall_score(y_true, y_pred)
print( 'Train Precision = ', model_precision, '\n', 'Train Recall = ', model_recall)
```

Call the function.

```
analyze_model_on_train_set(seq_set, RNN_fwd)
```

```
313/313 [=====] - 1s 2ms/step - loss: 0.1905 - accuracy: 0.9265
Train Accuracy: 0.9264922142028809
79/79 [=====] - 0s 2ms/step
Confusion matrix
- x-axis is true labels.
- y-axis is predicted labels
[[12117  414]
 [ 735 2365]]
Train Precision = 0.8510255487585462
Train Recall = 0.7629032258064516
```

```
def analyze_model_on_test_set(input_sequence_columns, model_path, width= 10, height=5):
```

We will consider all the last sequences of the test set.

```
last_test_seq = [test_df[test_df['id']==id][input_sequence_columns].values[-sequence_length:]
                  for id in test_df['id'].unique() if len(test_df[test_df['id']==id]) >= sequence_length]
last_test_seq = np.asarray(last_test_seq).astype(np.float32)
```

Get the labels of the test set.

```
y_mask = [len(test_df[test_df['id']==id]) >= sequence_length for id in test_df['id'].unique()]
last_test_label = test_df.groupby('id')['failure_within_w1'].nth(-1)[y_mask].values
last_test_label = last_test_label.reshape(last_test_label.shape[0],1).astype(np.float32)
```

If the best model weights were saved, load them.

```
if os.path.isfile(model_path):
    print("using " + model_path)
    model_estimator = load_model(model_path)
```

Test the metrics.

```
start = time.time()
scores_test = model_estimator.evaluate(last_test_seq, last_test_label, verbose=2)
end = time.time()
print("Total time taken for inferencing: ", "{:.2f}".format((end-start)), " secs")

print('Test Accuracy: {}'.format(scores_test[1]))
```

Make predictions and compute the confusion matrix.

```
y_model_estimator_pred_test = (model_estimator.predict(last_test_seq) > 0.5).astype("int32")
y_true_test = last_test_label
```

```
test_set = pd.DataFrame(y_model_estimator_pred_test)
test_set.to_csv('binary_submit_test.csv', index = None)
```

```
print('Confusion matrix\n- x-axis is true labels.\n- y-axis is predicted labels')
model_estimator_conf_m = confusion_matrix(y_true_test, y_model_estimator_pred_test)
print(model_estimator_conf_m)
```

#Compute the precision and recall values.

```
model_estimator_precision_test = precision_score(y_true_test, y_model_estimator_pred_test)
model_estimator_recall_test = recall_score(y_true_test, y_model_estimator_pred_test)
f1_test = 2 * (model_estimator_precision_test * model_estimator_recall_test) /
(model_estimator_precision_test + model_estimator_recall_test)
print('Test Precision: ', model_estimator_precision_test, '\n', 'Test Recall: ',
model_estimator_recall_test, '\n', 'Test F1-score:', f1_test )
```

#Plot the predicted data in blue and the

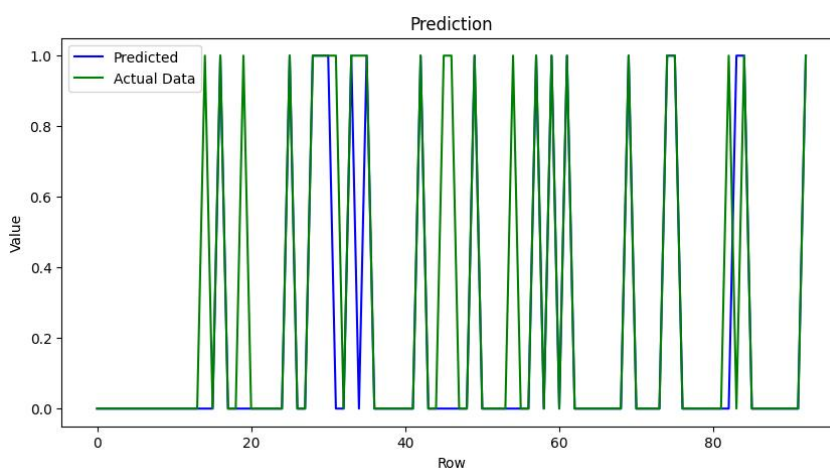
#actual data in green to visually verify the accuracy of the model.

```
fig_verify = plt.figure(figsize=(10, 5))
plt.plot(y_model_estimator_pred_test, color="blue")
plt.plot(y_true_test, color="green")
plt.title('Prediction')
plt.ylabel('Value')
plt.xlabel('Row')
plt.legend(['Predicted', 'Actual Data'], loc='upper left')
plt.show()
```

#Call the function

```
analyze_model_on_test_set(["s2"], RNN_fwd_path, 10, 5)

using /kaggle/working/RNN_fwd.h5
3/3 - 0s - loss: 0.2740 - accuracy: 0.9032 - 161ms/epoch - 54ms/step
Total time taken for inferencing: 0.19 secs
Test Accuracy: 0.9032257795333862
3/3 [=====] - 0s 4ms/step
Confusion matrix
- x-axis is true labels.
- y-axis is predicted labels
[[67  1]
 [ 8 17]]
Test Precision: 0.9444444444444444
Test Recall: 0.68
Test F1-score: 0.7906976744186047
```



{REST OF THE CODE}

...

~REFERENCE~

[1] *Deep Learning for Predictive Maintenance*

https://github.com/Azure/lstms_for_predictive_maintenance/blob/master/Deep%20Learning%20Basics%20for%20Predictive%20Maintenance.ipynb/

[2] *Predictive Maintenance: Step 2A of 3, train and evaluate regression models*

<https://gallery.cortanaintelligence.com/Experiment/Predictive-Maintenance-Step-2A-of-3-train-and-evaluate-regression-models-2/>

[3] A. Saxena and K. Goebel (2008). "Turbofan Engine Degradation Simulation Data Set", NASA Ames Prognostics Data Repository

<https://ti.arc.nasa.gov/tech/dash/groups/pcoe/prognostic-data-repository/#turbofan>

NASA Ames Research Center, Moffett Field, CA